Executive Summary: Authentication & Security

This audit report evaluates the MEP Calculator codebase against the FENMO (Globel) Application QA/UAT Checklist for the Authentication & Security domain (IDs 1, 2, 24, 27, and 28). It isolates exact code vulnerabilities, missing business rules, and provides automated drop-in remediation specifications for zero-ambiguity execution.

ID	Checklist Item	Severity	Audit Result	Action Required
ID 1	Login -> OTP -> Primary Policy Page	High	GAP DETECTED	Enforce Policy redirect in AuthGuard
ID 2	Forgot Password (No Previous Password Reuse)	High	DEFECT DETECTED	Add bcrypt comparison in reset-password
ID 24	Email Login - Device Binding & Licensing	High	PASS (100%)	None - HWID binding operational
ID 27	Device Login Persistence	High	PASS (100%)	None - 7d JWT & LocalStorage active
ID 28	Forgot Password - Reject Previous Password	High	DEFECT DETECTED	Add previous password rejection check

1. Detailed Defect Analysis & Root Causes

Defect Ref	Affected Component & Path	Root Cause & Impact
SEC-DEF-01 (IDs 2 & 28)	Backend Route Handler File: backend/authServer.js Endpoint: POST /api/auth/reset-password Lines: 1148 - 1180	ROOT CAUSE: The reset-password handler hashes the submitted new password and writes it to DB without comparing it against the existing password hash. IMPACT: A user can reset their password to their current password, violating the strict password change rule and UAT criteria.
SEC-DEF-02 (ID 1)	Frontend Route Guard File: frontend/src/routes/__root.tsx Component: AuthGuard Lines: 64 - 80	ROOT CAUSE: The route guard checks `isLoggedin` and `isAccountVerified`, but fails to evaluate `profile.privacyPolicyAccepted !== true`. IMPACT: Newly registered and verified users navigate straight to the workspace instead of being gated at the primary policy/EULA page (/privacy).

2. Automated Code Remediation Specifications

Remediation 1 (SEC-DEF-01): Prevent Previous Password Reuse in backend/authServer.js

Apply the following bcrypt comparison block inside the POST /api/auth/reset-password handler:

```
// In backend/authServer.js (around Line 1170):
if (user.password) {
  const isSameAsOld = await bcrypt.compare(newPassword, user.password);
  if (isSameAsOld) {
    return sendJSON(res, 200, {
      success: false,
      message: 'New password cannot be the same as your previous password. Please choose a different password.'
    });
  }
}
const hashedPassword = await bcrypt.hash(newPassword, 10);
user.password = hashedPassword;
user.resetOtp = '';
user.resetOtpExpireAt = 0;
```

Remediation 2 (SEC-DEF-02): Enforce Primary Policy Route Gating in frontend/src/routes/__root.tsx

Apply the following route guard check inside AuthGuard component in frontend/src/routes/__root.tsx:

```
// In frontend/src/routes/__root.tsx inside function AuthGuard({ children }):
const isPrivacyPage = routerState.location.pathname === '/privacy';

// Enforce Primary Policy (EULA) Acceptance for active authenticated sessions
if (session && profile && !isLoginPage && !isPrivacyPage) {
  const hasAccepted = profile.privacyPolicyAccepted === true ||
    (typeof window !== 'undefined' && localStorage.getItem('privacy_policy_accepted') === 'true');
  if (!hasAccepted) {
    return <Navigate to="/privacy" replace />;
  }
}
```

3. Automated Execution & Verification Protocol

When this PDF report is presented for execution, Antigravity will automatically:
1. Apply the bcrypt previous password check to backend/authServer.js.
2. Integrate the privacy policy route guard to frontend/src/routes/__root.tsx.
3. Perform live end-to-end testing (Reset password with old vs new password, and verify /privacy gating on fresh signup).
4. Mark Checklist IDs 1, 2, and 28 as 100% PASS in the QA Matrix.